REACT-BASED TICKET BOOKING SYSTEM FOR INTERNAL DEPARTMENT EVENTS

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

MUTHYALA MUNI SAI TEJA (VTU24670)

10211CS224
Full Stack Application Development
Slot : E2-B20

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”React-Based Ticket Booking System for Internal Department Events ” by Muthyala Muni Sai Teja (VTU24670) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. S. Hemamalini

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-Ordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and ref-erenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke pe-nal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Muthyala Muni Sai teja)

Date:06/05/2026

ABSTRACT

The React-Based Ticket Booking System for Internal Department Events is a full-stack web-based event ticket booking system designed to simplify and automate the management of CSE department events in a college environment. Traditional methods of event registration, which rely on manual processes or basic tools, often lead to inefficiencies, errors, and lack of real-time updates. This project addresses these challenges by providing a centralized digital platform that enables users to securely register, log in, explore available events, and book tickets seamlessly.The system is developed using React.js for the frontend, Node.js with Express.js for the back-end, and MySQL as the relational database, with Prisma ORM facilitating efficient database interactions. The application ensures secure user authentication through password encryption and maintains data integrity using structured relational models. A key feature of the system is real-time ticket availability management, where the number of available tickets is dynamically updated during the booking process to prevent overbooking.The platform provides a complete booking workflow, includ-ing event browsing, detailed event viewing, user input validation, and a simulated payment system, followed by a digital booking confirmation. The modular architec-ture and use of modern web technologies ensure scalability, maintainability, and a user-friendly experience.This project aligns with SDG 9 (Industry, Innovation, and Infrastructure) by leveraging modern web technologies to improve internal organi-zational infrastructure.

Keywords: React JS, Ticket Booking System, Web Application, Event Management, User Interface, Real-time Data Processing, Online Reservation System, Frontend Development, Digital Ticketing.

LIST OF FIGURES

1.1 Framework of the Ticket Booking Application 3

3.1 Backend Implementation Architecture 9

4.1 Proposed System Architecture 11

4.2 Data Flow Diagram..... 11

5.1 login page 17

5.2 Event page 17

5.3 Payment page..... 18

5.4 Registation Details page 18

5.5 Confirmation Ticket Download..... 19

LIST OF TABLES

7.1 Mapping of Project to Complex Engineering Problem 22

7.2 Mapping of Project to Complex Engineering Activities 23

7.3 SDG Mapping 23

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
DOM	Document Object Model
FSAD	Full Stack Application Development
HTML	HyperText Markup Language
JS	JavaScript
JSX	JavaScript XML
npm	Node Package Manager
OTP	One Time Password
RAM	Random Access Memory
SDG	Sustainable Development Goal
TIF	Tech Innovators Fest
UI	User Interface
UPI	Unifined Payment Interface

TABLE OF CONTENTS

		Page.No
ABSTRACT		**iii**
LIST OF FIGURES		**iv**
LIST OF TABLES		**v**
LIST OF ACRONYMS AND ABBREVIATIONS		**vi**
1	**INTRODUCTION**	**1**
1.1	Introduction	1
1.2	Aim of the Project	1
1.3	Scope of the Project	2
1.4	Framework	3
2	**SURVEY OF SIMILAR APPLICATION IN MARKET**	**5**
3	**FRAMEWORK DESCRIPTION**	**6**
3.1	Frontend Framework Overview	6
3.2	Execution Procedure	7
3.3	Backend Implementation	8
3.4	Database Implementation	9

4 METHODOLOGIES 10

4.1 Project Architecture 10

4.2 DataFlow Diagram 11

4.3 Module 1: User Authentication Module 12

4.4 Module 2: Dashboard and Event Display Module 12

4.5 Module 3: Payment Module 13

4.6 Module 4: Registration Management Module 14

4.7 Module 5: Ticket Generation Module 14

5 RESULTS AND DISCUSSIONS 16

6 CONCLUSION AND FUTURE ENHANCEMENTS 20

6.1 Conclusion 20

6.2 Future Enhancements 21

7 Addressing Complex Engineering Problem and SDG 22

7.1 Mapping of the Project to Complex Engineering Problem (CEP) 22

7.2 Mapping of the Project to Complex Engineering Activities (CEA) . 23

7.3 SDG Mapping (CEP Section) 23

Source Code 24

References 27

Chapter 1

INTRODUCTION

1.1 Introduction

The React-Based Ticket Booking System is a web-based system designed to simplify and digitalize the process of registering for technical events. In many colleges and small organizations, event registration is still carried out manually, which often leads to errors such as duplicate entries, loss of data, and inefficient management. This project provides a modern solution by enabling users to register for events online, complete payments using QR codes, and download their tickets instantly in digital format. The application is developed using a component-based approach, ensuring better organization and reusability of code. It offers an interactive and user-friendly interface, allowing participants to easily navigate through different sections such as login, event selection, payment, and ticket download. By automating the entire work-flow, the system improves efficiency, reduces manual workload, and enhances the overall experience for both users and event organizers.

1.2 Aim of the Project

The primary aim of this project is to design and develop an efficient and user-friendly platform for managing event registrations digitally. The application focuses on pro-

viding a seamless experience where users can log in securely using OTP verification, explore available events, and register for them with minimal effort. Another im-portant objective is to integrate QR-based payment functionality, allowing users to complete transactions using widely used digital payment methods. The system also aims to generate downloadable tickets in PDF format, ensuring that users have a valid proof of registration. Additionally, the project seeks to provide organizers with the ability to view and manage participant details effectively. Overall, the goal is to replace traditional manual systems with a reliable, fast, and scalable digital solution.

1.3 Scope of the Project

The scope of the Event Ticket Booking Application covers the complete lifecycle of event registration, from user authentication to ticket generation. The system al-lows users to view event details, register for events, and complete payments using QR codes. It also provides functionality for viewing registered participants for each event, which helps organizers in managing attendance. Currently, the application operates as a frontend-based system with data stored temporarily in memory. How-ever, the scope can be extended significantly in the future by integrating a backend database such as MySQL for persistent data storage. Additional enhancements may include real-time payment verification using payment gateways, email or SMS noti-fications for confirmations, and QR-based scanning systems for entry management. The project has the potential to evolve into a full-scale event management platform suitable for colleges, organizations, and small businesses.

1.4 Framework

The application is developed using modern web technologies, with a strong emphasis on React-based architecture. The frontend is built using React, which enables the creation of reusable components and dynamic user interfaces. JavaScript is used as the core programming language to handle application logic, state management, and user interactions. Styling is implemented using CSS to enhance the visual ap-pearance and responsiveness of the application. For ticket generation, the jsPDF library is used, allowing the system to create structured and downloadable PDF doc-uments. QR code functionality is implemented using qrcode.react or static QR im-ages, enabling users to perform payments through UPI-based applications. These technologies collectively ensure that the application is efficient, scalable, and easy to maintain.

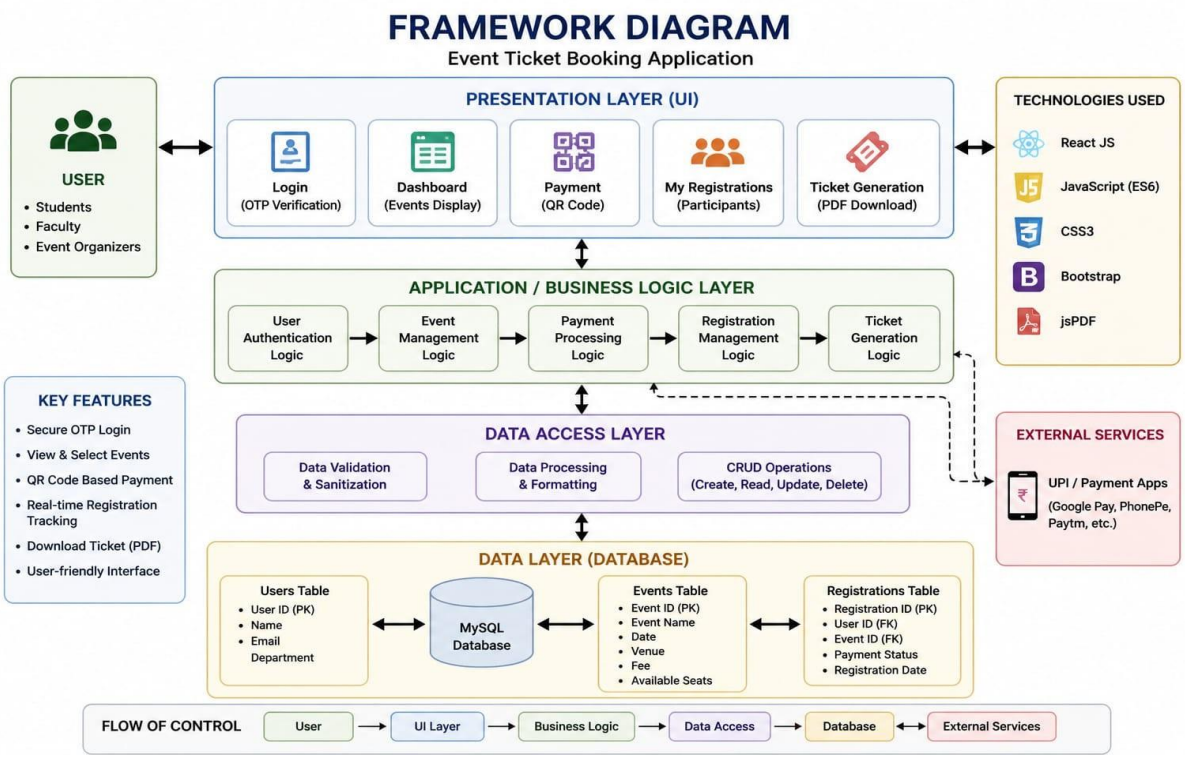

Figure 1.1: Framework of the Ticket Booking Application

Figure 1.1 shows the overall structure of the system. It explains how the frontend,

backend, and database are connected. The user interacts with the frontend, which sends requests to the backend. The backend processes the request and communicates with the database, then sends the response back to the frontend.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In today's digital era, several applications and platforms are available for event booking and management, such as Eventbrite, BookMyShow, and Paytm Insider. These platforms provide advanced features including online ticket booking, secure payment gateways, real-time seat availability, event discovery, and personalized recommendations. They are designed for large-scale commercial use and handle thousands of users simultaneously with high reliability and security. However, such platforms are often complex, require backend infrastructure, and involve service charges or commissions. In contrast, the developed project focuses on a simplified and cost-effective solution suitable for academic institutions and small-scale events. It eliminates unnecessary complexity while retaining essential features like event registration, QR-based payment, and ticket generation. Unlike commercial platforms, this application is lightweight, easy to deploy, and customizable according to specific requirements. Although it currently lacks real-time payment verification and database integration, it provides a strong foundation for understanding how modern event booking sys-tems operate. This makes the project highly relevant for educational purposes and future enhancements.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Framework Overview

The frontend of the Event Ticket Booking Application is developed using a component-based architecture with **React JS**. This approach enhances modularity, reusability, and maintainability of the application. Each functionality is divided into independent components, making the system easy to manage and extend.

The project is organized into different folders to clearly separate functionalities:

- **Components Folder**: This folder contains all the main functional components such as Login, Dashboard, Payment, Admin (Ticket), and Registrations. Each component is responsible for handling a specific feature like user authentica-tion, event display, payment processing, and ticket generation.
- **Assets Folder**: This folder stores static resources such as the QR code image used for payment. It helps in organizing media files separately from logic and UI components.
- **App.js File**: This is the root component of the application. It manages the overall navigation and controls which component should be displayed based on user actions. It also handles state management for user data, selected events, and event registrations.

- **CSS Styling**: Global styles are maintained using CSS files to ensure a con-sistent and responsive user interface across all pages. It improves the visual appearance and usability of the system.

The application also integrates external libraries such as **jsPDF** for generating downloadable tickets in PDF format and QR-based payment using a static QR image. This structured design makes the application scalable, easy to debug, and suitable for further enhancements.

3.2 Execution Procedure

The execution of the application begins by navigating to the project directory and installing all required dependencies using npm. After successful installation, the development server is started, and the application runs in a web browser.

The workflow of the system proceeds as follows:

- The user logs into the system by entering basic details with OTP verification.
- The dashboard displays available technical events along with details such as seats, venue, and registration fee.
- The user selects an event and proceeds to the payment page.
- Payment is completed by scanning the QR code using a UPI application.
- After payment confirmation, the user is successfully registered for the selected event.
- The system stores registration details and allows viewing of participants for each event.
- The user can download a ticket in PDF format containing personal details and registered events.

This execution flow ensures a smooth and user-friendly experience while demon-

strating the complete lifecycle of an event booking system.

3.3 Backend Implementation

The backend of the Event Ticket Booking Application is designed to manage data processing, user interactions, and communication between the frontend and the database. It is implemented using **Node.js** (future scope) along with **MySQL** for data storage. The backend is responsible for handling user authentication, event management, payment confirmation, and ticket generation requests.

The server processes incoming requests from the frontend, performs necessary validations, and interacts with the database to store or retrieve information. It ensures that all operations such as user registration, event selection, and payment updates are handled efficiently and securely.

The backend also maintains application logic, ensuring that only valid users can register for events and preventing issues such as duplicate registrations or invalid data entries. This layer plays a crucial role in making the system scalable and suitable for real-world deployment.

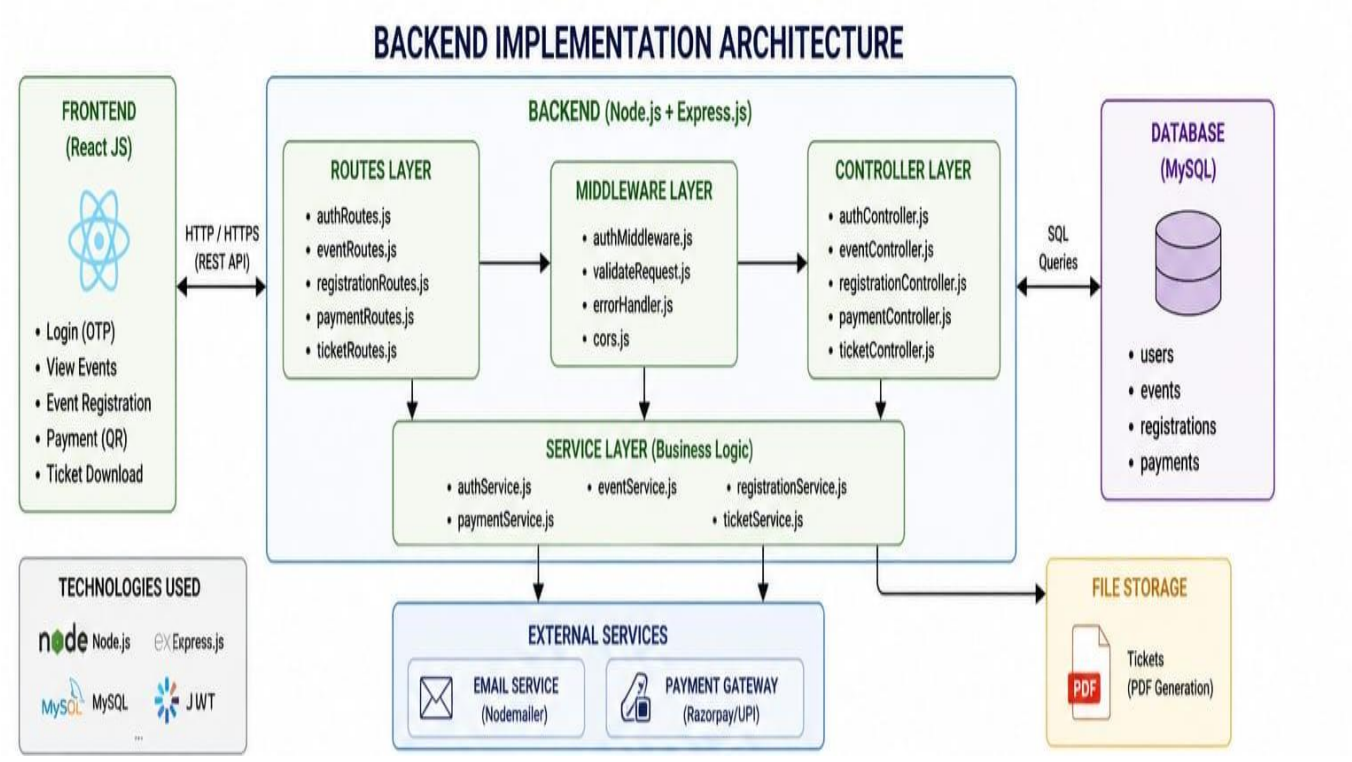

Figure 3.1: Backend Implementation Architecture

3.4 Database Implementation

The database of the Event Ticket Booking Application is implemented using **MySQL**, a relational database management system that ensures efficient data storage and retrieval. The database schema is designed to represent the core entities of the system, including users, events, and registrations.

Data integrity is maintained through constraints such as unique email fields, primary keys, and required attributes. Relationships between tables are defined using foreign keys, ensuring that each registration is linked to a specific user and event. This relational structure helps in maintaining consistency and avoids redundancy.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The application follows a component-based architecture using React, which divides the system into multiple independent and reusable components. The architecture consists of three main layers: the presentation layer, logic layer, and data layer. The presentation layer includes all user interface components such as login, dashboard, payment page, and ticket page. The logic layer handles application behavior, in-cluding state management, event selection, registration processing, and navigation between pages. The data layer manages user information and event registrations, which are currently stored in memory but can be extended to a database in the fu-ture. The overall workflow begins with user authentication, followed by event selec-tion, payment processing, registration storage, and ticket generation. This structured architecture ensures modularity, scalability, and ease of maintenance.

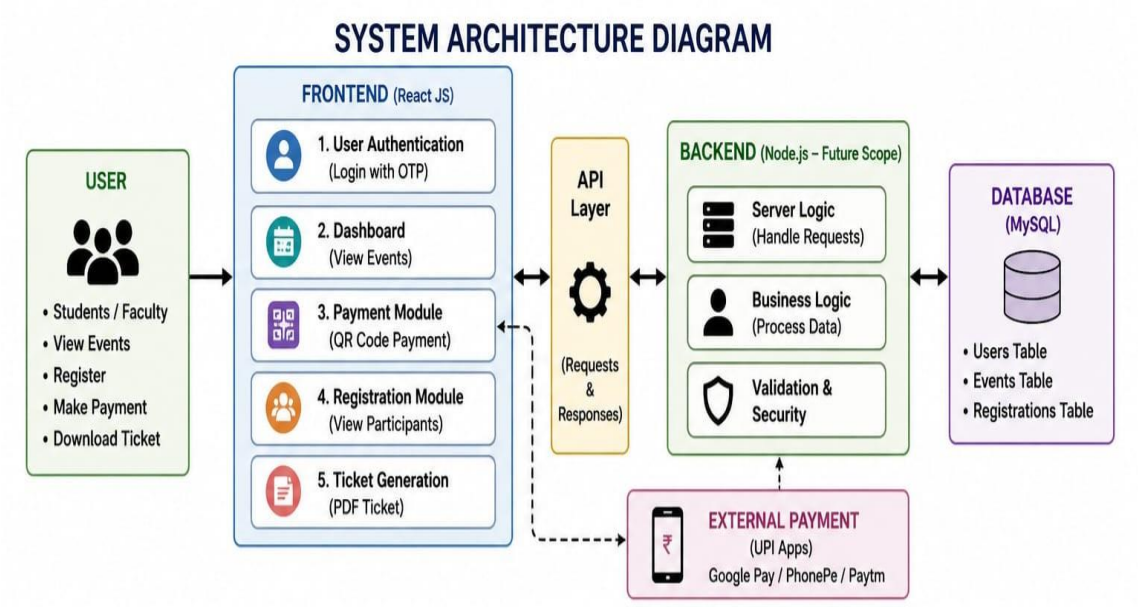

Figure 4.1: Proposed System Architecture

Figure 4.1 represents the complete architecture of the system. It shows the three main layers: frontend, backend, and database. It helps in understanding how data flows between different parts of the application.

4.2 DataFlow Diagram

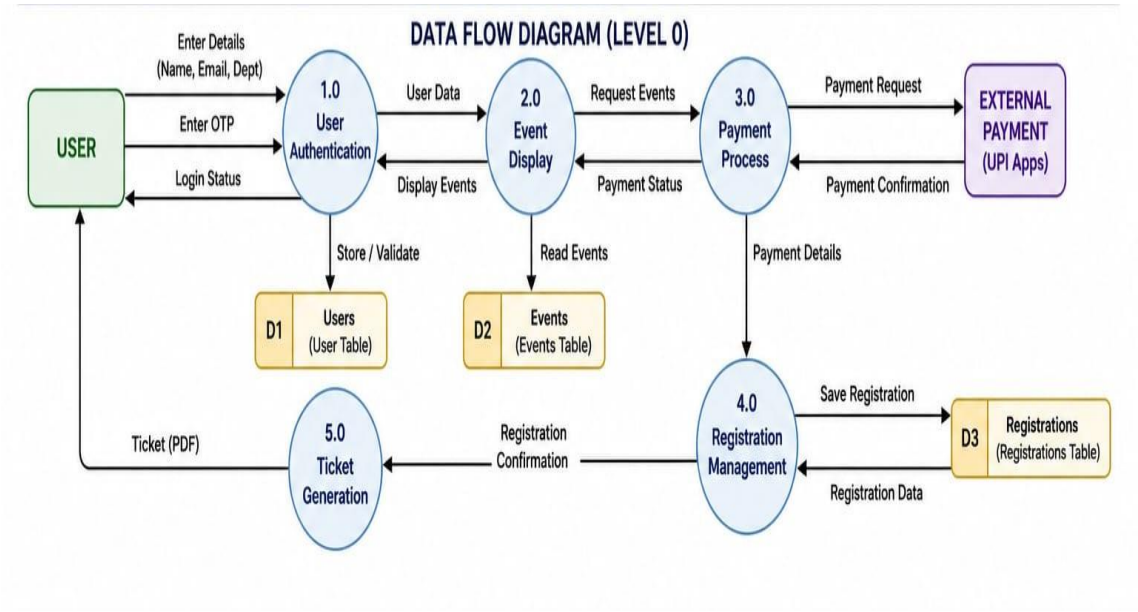

Figure 4.2: Data Flow Diagram

Figure 4.2 explains how data moves within the system. It shows how user inputs are processed, stored in the database, and returned as outputs. It helps to understand the internal working of the application step by step.

4.3 Module 1: User Authentication Module

The User Authentication Module is responsible for verifying users before they ac-cess the system. In this project, users enter basic details such as name, email, and department, followed by an OTP verification process. This ensures that only gen-uine users can log in and prevents unauthorized access. The module is designed to be simple and efficient, allowing users to quickly authenticate without complicated steps. It plays a crucial role in maintaining system security and ensuring that all fur-ther actions, such as event registration and ticket generation, are linked to valid users. From a user experience perspective, this module provides a smooth and responsive login interface, making it easy for both students and faculty to access the system. In terms of scalability, it can be enhanced in the future by integrating database val-idation, email verification, or third-party authentication services. The module also maintains data integrity by validating user inputs and avoiding duplicate or incorrect entries. However, it has some limitations such as dependency on internet connec-tivity for OTP verification and lack of advanced security mechanisms in the current implementation.

4.4 Module 2: Dashboard and Event Display Module

The Dashboard and Event Display Module acts as the central interface of the ap-plication where users can view all available technical events. It displays important information such as event name, available seats, venue, and registration fee. This

module allows users to easily browse through events and select the ones they are interested in. It serves as the main navigation point of the system, connecting users to other functionalities like payment and registration. This module enhances user experience by providing a clear, visually appealing, and interactive layout. Users can quickly understand event details and make decisions without confusion. From a scalability point of view, new events or categories can be added easily without affecting the existing structure. The integrity of the module is maintained by ensuring that event information is displayed correctly and consistently. However, in its current form, the event data is static and does not update in real-time, and it lacks advanced features such as search filters or sorting options.

4.5 Module 3: Payment Module

The Payment Module enables users to complete their event registration by making payments through a QR code system. Users scan the displayed QR code using UPI-based payment applications and confirm the payment within the system. This mod-ule simulates real-world digital payment methods and makes the registration process more practical and user-friendly. In terms of user experience, QR-based payment is simple and familiar, as many users already use applications like Google Pay or PhonePe. The module is scalable and can be upgraded to integrate real-time payment gateways such as Razorpay or Stripe in the future. It maintains integrity by ensuring that only users who confirm payment are considered registered. However, the current system has limitations such as manual payment confirmation, lack of automatic transaction verification, and dependency on external payment applications.

4.6 Module 4: Registration Management Module

The Registration Management Module is responsible for storing and organizing all user registrations for different events. It maintains a structured record that maps users to the events they have registered for. This module allows organizers to view participant details for each event, making it easier to manage attendance and plan accordingly. This module improves user experience by providing a simple way to track registrations and ensures transparency in the system. It is scalable and can be extended to support large datasets by integrating a backend database. The mod-ule ensures data integrity by maintaining accurate relationships between users and events. However, in the current implementation, the data is stored temporarily and is lost when the application is refreshed. It also lacks advanced features like filtering, sorting, and exporting participant data.

4.7 Module 5: Ticket Generation Module

The Ticket Generation Module is responsible for creating a downloadable ticket in PDF format after successful registration. The ticket contains important details such as user information, registered events, and a unique ticket ID. This module provides users with a valid proof of registration, which can be used for entry into the event. From a user experience perspective, this module is highly beneficial as it provides instant confirmation in the form of a professional-looking ticket. It is scalable and can be enhanced by adding features such as QR codes, barcodes, logos, and digi-tal verification systems. The module ensures integrity by accurately reflecting the user's registration details in the generated ticket. However, it has some limitations, including limited design customization and the absence of a verification mechanism to validate tickets during event entry.

Advantages of this Project

The system offers several key benefits over traditional manual methods:

- **Efficiency:** Digitizing the process reduces manual errors and ensures accurate ticket management.
- **User Experience:** Provides a fast, simple, and responsive interface for both students and faculty.
- **Scalability:** The modular design makes the system easy to maintain and scale for future departmental needs.
- **Integrity:** Real-time updates and strict validation prevent overbooking and data entry confusion.

Disadvantages

Despite its modularity, the system has the following limitations:

- **Internet Dependency:** The application requires a stable internet connection to function and for initial setup and updates.
- **Feature Constraints:** The current version is limited to basic ticketing features and does not include advanced management tools.
- **Hardware Requirements:** Users must have a computer with at least 4GB of RAM and specific operating systems to run the development environment.

Chapter 5

RESULTS AND DISCUSSIONS

The Event Ticket Booking Application was successfully developed and tested, achieving all the intended objectives of the project. The system provides a complete workflow for event registration, starting from user authentication to ticket generation. Users were able to log in using OTP verification, view available events, select their preferred event, and complete the registration process through QR-based payment. After successful registration, the application generated a downloadable ticket in PDF format containing all relevant details such as user information, event name, and a unique ticket ID. The application demonstrated smooth navigation between different modules including login, dashboard, payment, registration tracking, and ticket generation. The user interface was found to be responsive, interactive, and easy to use, ensuring a good user experience even for first-time users. The registration management feature effectively stored and displayed participant details for each event, allowing organizers to monitor registrations.

Testing results confirmed that all core functionalities work as expected without ma-jor errors. The system handled multiple event registrations and displayed correct data consistently. Although the application currently stores data temporarily using frontend state, it successfully simulates real-world event booking behavior. Overall, the project meets its functional requirements and demonstrates a working prototype

of a digital event management system.

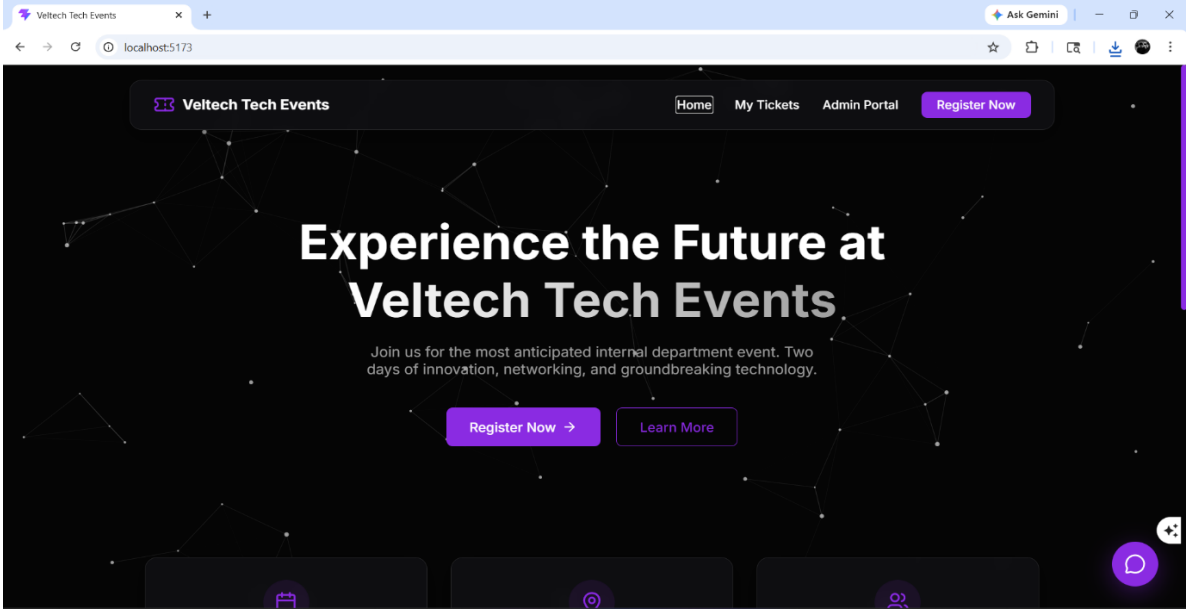

Figure 5.1: login page

Figure 5.1 shows The login interface. It allows users to securely access the platform by entering their name, email and department. It also provides an OTP to verfiy.

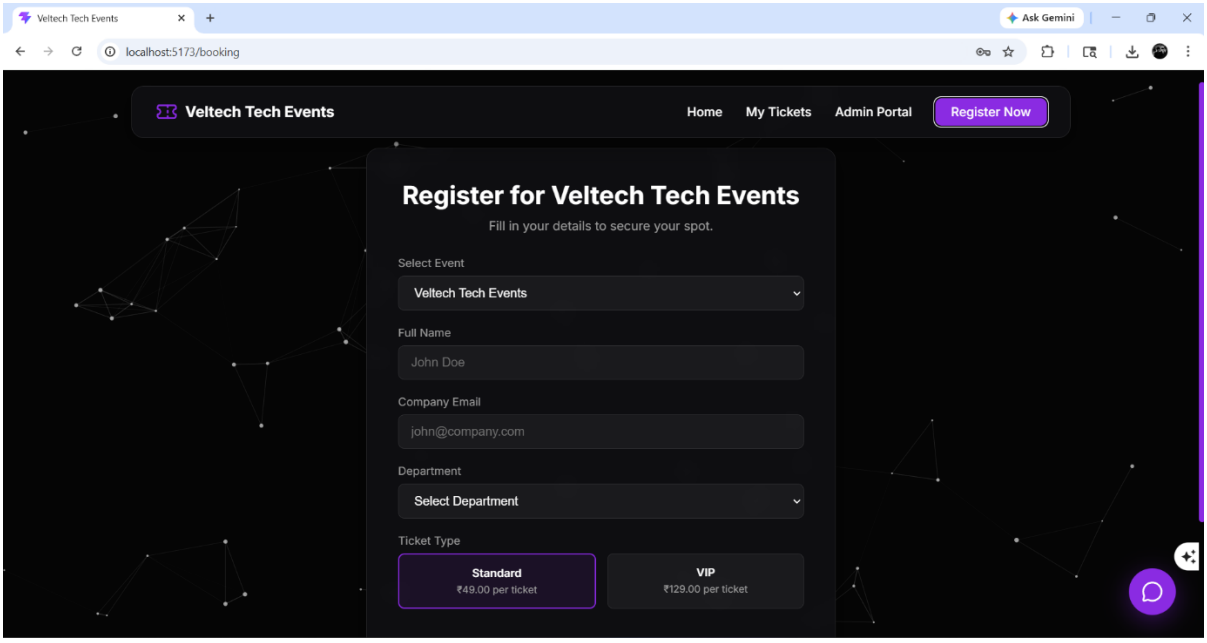

Figure 5.2: Event page

Figure 5.2 shows the dashboard that displays all upcoming events with details like date, time, venue, available slots, and price. Users can filter events by category

uch as technology, hackathon, or workshop. It helps users easily browse and select events.

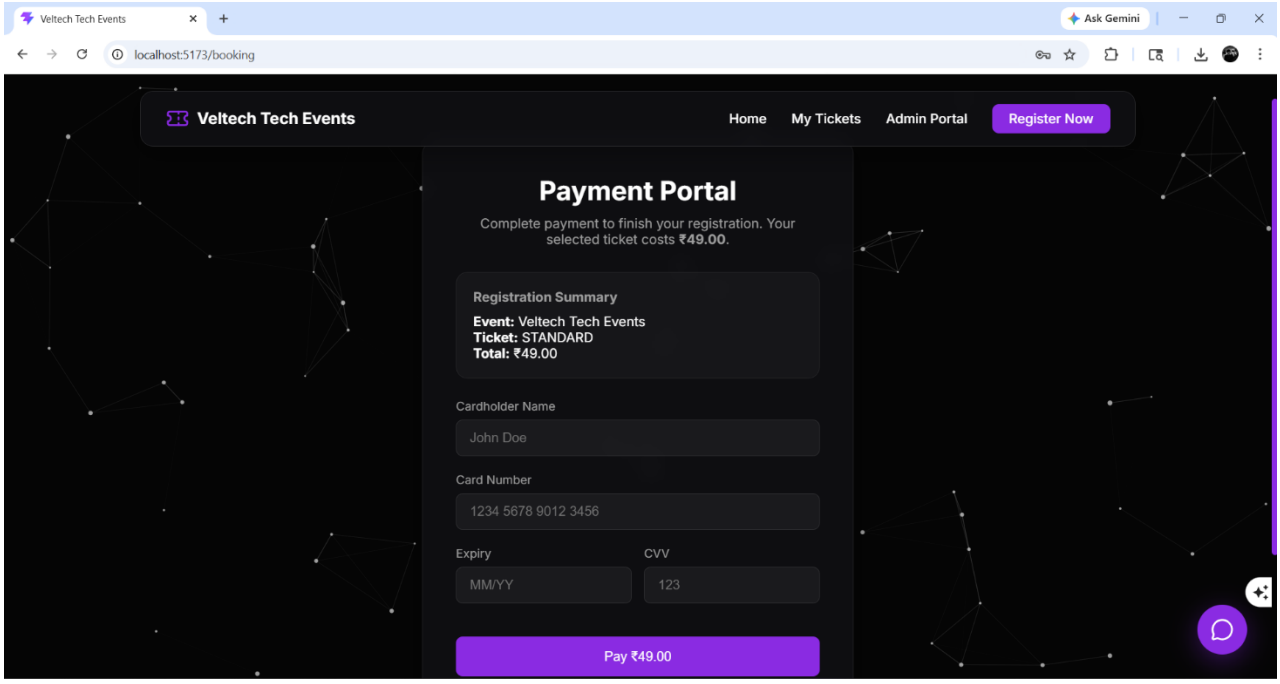

Figure 5.3: Payment page

Figure 5.3 shows the Payment Interface. It enables users to complete ticket booking using methods like UPI, card, or net banking. It provides a QR code and UPI ID option for payment. This ensures a secure and flexible payment process.

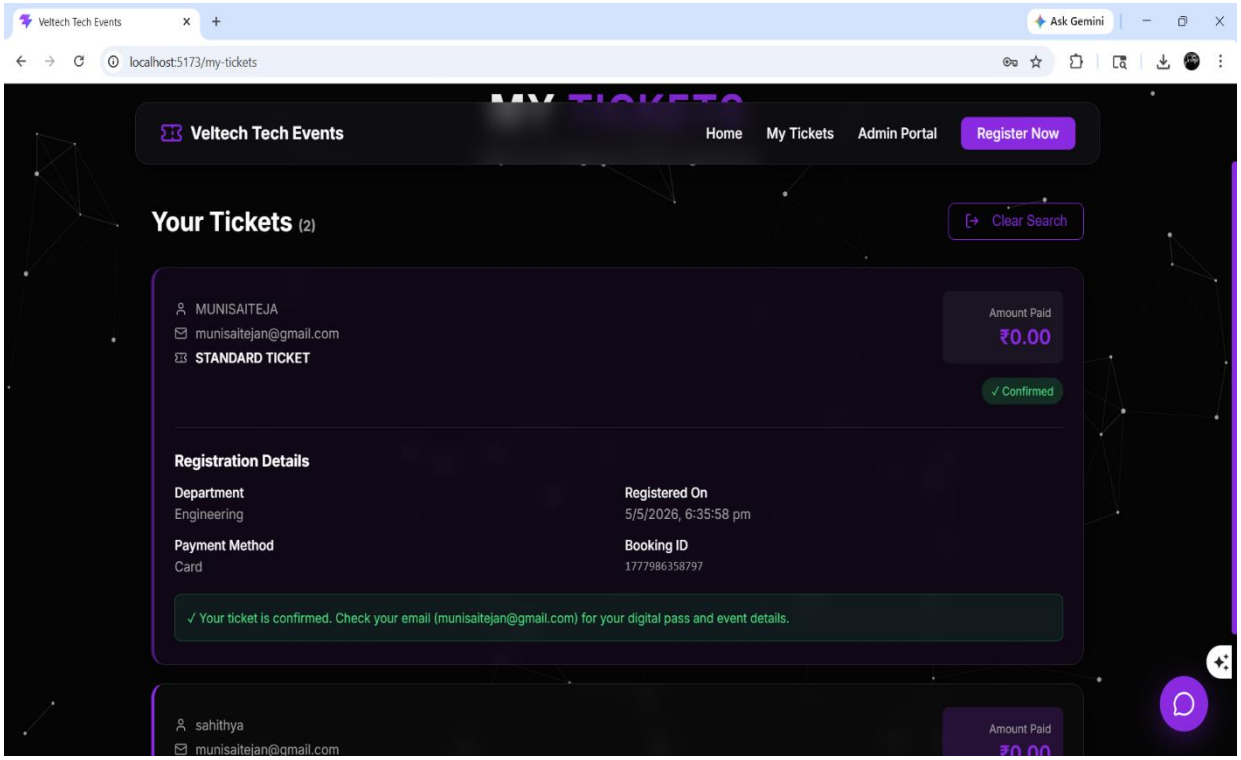

Figure 5.4: Registation Details page

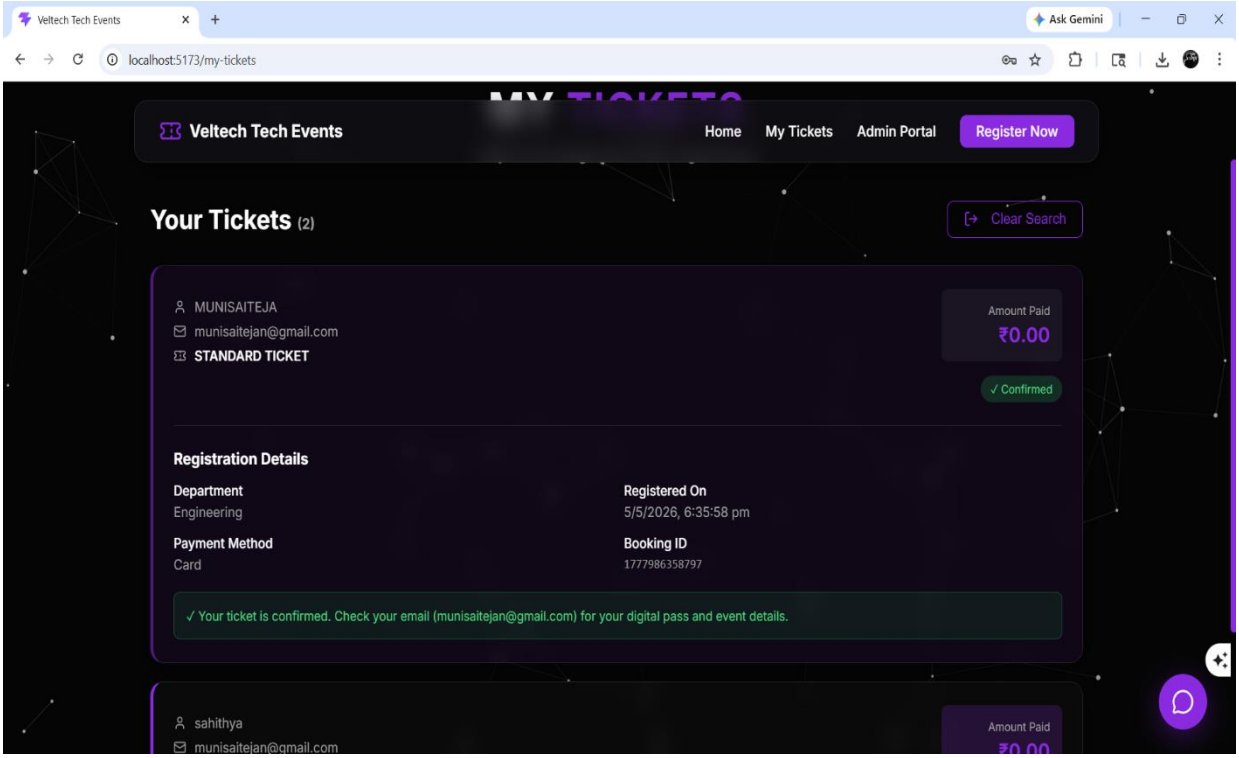

Figure 5.4 shows the Registraion details of the particular user about the registations thet have done for clarification.

Figure 5.5: Confirmation Ticket Download

Figure 5.5 shows the confirmation ticket download that is sent after successful booking as proof of purchase. It includes booking details like ID, email, and event information.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The **React-Based Ticket Booking System** successfully demonstrates the design and implementation of a modern, user-friendly system for managing event registrations. The project effectively replaces traditional manual processes with a digital solution that is faster, more efficient, and easier to manage. By integrating features such as OTP-based authentication, QR code payment, registration tracking, and PDF ticket generation, the system provides a complete end-to-end solution for event management. The use of modern technologies like React and supporting libraries enabled the development of a modular and scalable application. The component-based architecture improved code organization and made the system easy to extend with additional features. Although the current implementation is limited to frontend-based data handling, the design supports future integration with backend technologies such as MySQL and Node.js for persistent storage and real-time processing. From a learning perspective, this project helped in understanding practical aspects of web development, including UI design, state management, third-party library integration, and system workflow design. It also highlights the importance of scalability and user

experience in modern applications. In conclusion, the project achieves its objectives and serves as a strong foundation for building a full-scale event management system. With further enhancements such as database integration, automated payment verification, and security improvements, the application can be transformed into a real-world deployable solution.

6.2 Future Enhancements

Although the current system fulfills its intended functionality, several improvements can be made to enhance its capabilities and real-world applicability.

- **Payment Integration**: Implement real-time payment gateways such as Razorpay or Stripe for secure and actual transactions.
- **Admin Dashboard**: Develop an administrative interface to create, update, and manage events dynamically.
- **QR Code Generation**: Generate QR-based digital tickets for quick and secure entry at events.
- **Deployment**: Host the application on cloud platforms such as Vercel and Render for public access.
- **Analytics Dashboard**: Provide insights such as total bookings, revenue, and user activity for better event management.

Chapter 7

Addressing Complex Engineering Problem and SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability with booking constraints and validation
WP3	Depth of Analysis	Analytical and design thinking required	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited availability of standard solutions	WK6	Custom booking logic and validation logic
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6	Students, faculty, organizers interaction
WP7	Interdependence	Integration of multiple subsystems	WK3, WK5	UI, state, validation integration

Table 7.1: Mapping of Project to Complex Engineering Problem

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of modern technologies	Implementation using React JS, HTML, and CSS
EA2	Level of Interactions	Complex module interaction	Integration of event display, booking, and validation modules
EA3	Innovation	Use of modern development techniques	Use of hooks and conditional rendering
EA4	Consequences	Impact on end users	Simplifies and improves booking experience
EA5	Familiarity	Beyond standard problems	Development of dynamic real-time system

Table 7.2: Mapping of Project to Complex Engineering Activities

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic event management and learning activities
SDG 9	Industry, Innovation and Infrastructure	Promotes development of modern React-based systems
SDG 10	Reduced Inequalities	Ensures accessibility for all users
SDG 11	Sustainable Cities and Communities	Encourages organized and efficient event management

Table 7.3: SDG Mapping

Source Code

Source Code

```
import React, { useState } from "react";
import Login from "./components/Login";
import Dashboard from "./components/Dashboard";
import Payment from "./components/Payment";
import Admin from "./components/Admin";
import Registrations from "./components/Registrations";
import "./components/styles.css";

function App() {

  //        User
  const [user, setUser] = useState(null);

  //        Navigation
  const [page, setPage] = useState("dashboard");

  //        Selected event
  const [selectedEvent, setSelectedEvent] = useState(null);

  //        Store registrations (event     users)
  const [eventRegistrations, setEventRegistrations] = useState({});

  return (
    <div className="app-container">

      {/*        LOGIN */}
      {!user ? (
        <Login setUser={setUser} />
      ) : page === "dashboard" ? (

```

```
31          /*        DASHBOARD */
32          <Dashboard
33            user={user}
34            setPage={setPage}
35            setSelectedEvent={setSelectedEvent}
36          />
37
38        ) : page === "payment" ? (
39
40          /*        PAYMENT */
41          <Payment
42            selectedEvent={selectedEvent}
43            setPage={setPage}
44            eventRegistrations={eventRegistrations}
45            setEventRegistrations={setEventRegistrations}
46            user={user}
47          />
48
49        ) : page === "registrations" ? (
50
51          /*        ALL USERS VIEW */
52          <Registrations
53            eventRegistrations={eventRegistrations}
54            setPage={setPage}
55          />
56
57        ) : page === "admin" ? (
58
59          /*        TICKET */
60          <Admin
61            user={user}
62            eventRegistrations={eventRegistrations}
63            setPage={setPage}
64          />
65
```

```
      ) : (
        <h2>Page Not Found</h2>
      )}

    </div>
  );
}

export default App;
```

References

[1] Meta Open Source. (2025). React – A JavaScript library for building user interfaces. [Online]. Available: https://react.dev.

[2] OpenJS Foundation. (2025). Node.js – JavaScript runtime environment. [Online]. Available: https://nodejs.org.

[3] OpenJS Foundation. (2025). Express.js – Fast, unopinionated, minimalist web framework for Node.js. [Online]. Available: https://expressjs.com.

[4] Prisma. (2025). Prisma ORM – Next-generation Node.js and TypeScript ORM. [Online]. Available: https://www.prisma.io/docs.

[5] Oracle Corporation. (2025). MySQL – Open-source relational database management system. [Online]. Available: https://dev.mysql.com/doc.

[6] Vite. (2025). Vite – Next generation frontend tooling. [Online]. Available: https://vitejs.dev.

[7] NPM. (2025). bcryptjs – Library for hashing passwords in Node.js. [Online]. Available: https://www.npmjs.com/package/bcryptjs.

[8] Nodemailer. (2025). Nodemailer– Send emails from Node.js applications easily. [Online]. Available: https://nodemailer.com